INSTITUTO FEDERAL
Santa Catarina

Departamento Acadêmico de Eletrônica – **DAELN**
IFSC Câmpus Florianópolis

Programação C++

Introdução à linguagem C++

Prof. Matheus Leitzke Pinto
matheus.pinto@ifsc.edu.br

Introdução

OBJETIVOS DA DISCIPLINA

- Conhecer os fundamentos da estrutura, semântica e sintaxe da linguagem C++.
- Conhecer a codificação das instruções.
- Conhecer ferramentas de desenvolvimento, depuração e documentação de software em linguagem C++: compiladores, ambientes integrados de desenvolvimento e ferramentas de depuração.
- Selecionar adequadamente instruções e funções de biblioteca da linguagem C++ para desenvolvimento de software, principalmente para sistemas embarcados e microcontrolados.

Sumário de aula

- Origem da linguagem C++ e aplicações
- Forma geral de um programa em C++
- Etapas de desenvolvimento de um programa
- Conteúdo programado
- Avaliação na disciplina

INSTITUTO FEDERAL
Santa Catarina

DAELN

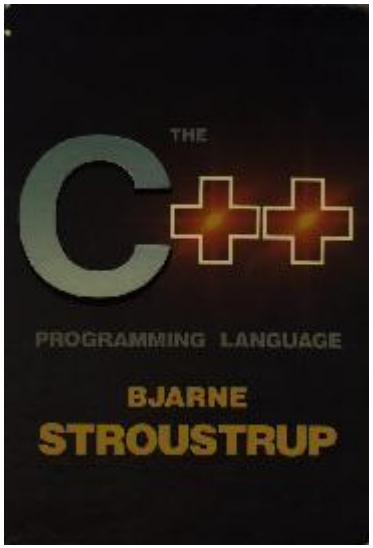
Introdução à linguagem C++

Origem da linguagem C++ e aplicações

Origem da linguagem C++ e aplicações

- Linguagem C++ criada em 1979 durante tese de doutorado

Primeira referência da linguagem (1985)

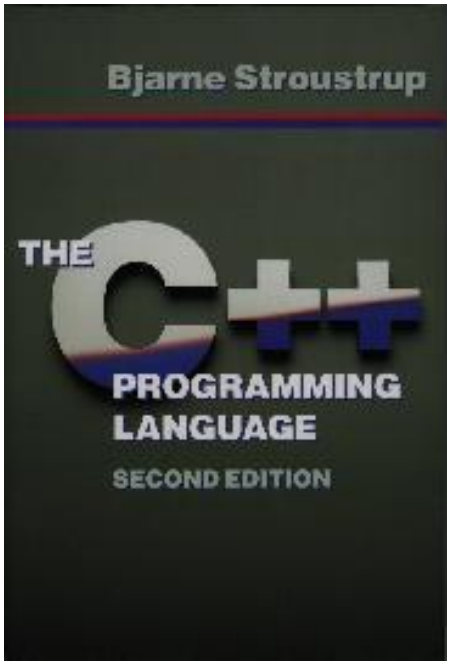


- Objetivo: ser um C melhorado

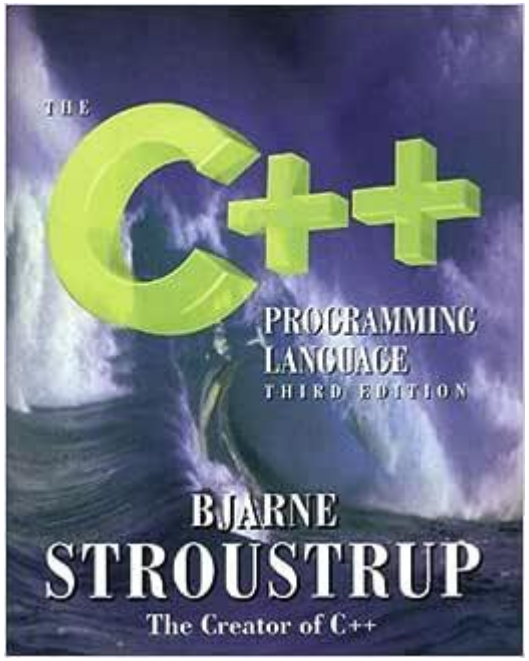
Criador: Bjarne Stroustrup

Origem da linguagem C++ e aplicações

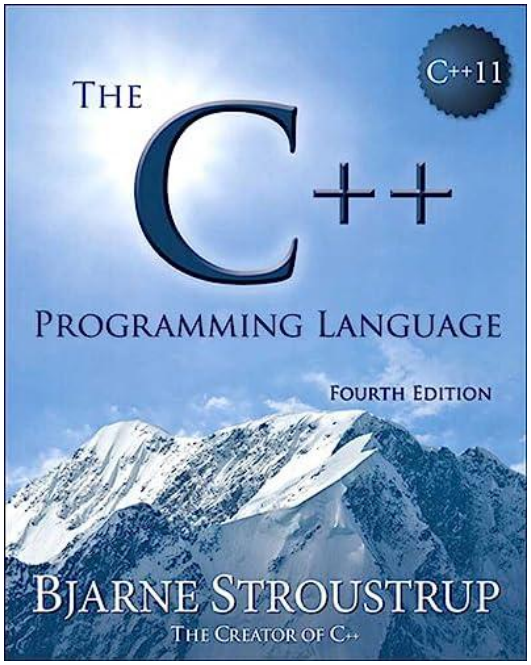
Bjarne fez mais edições do seu livro...



(1991)



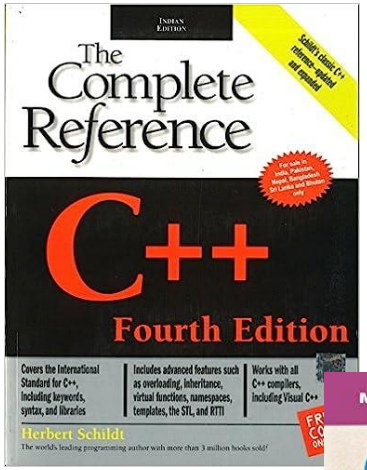
(1997)

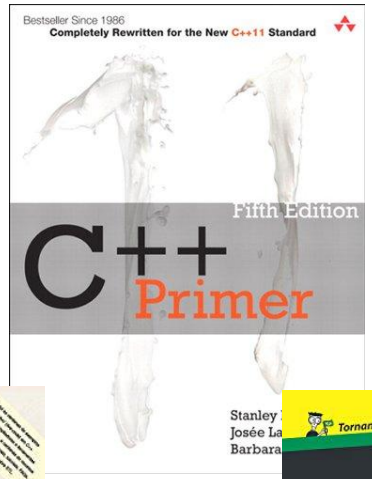


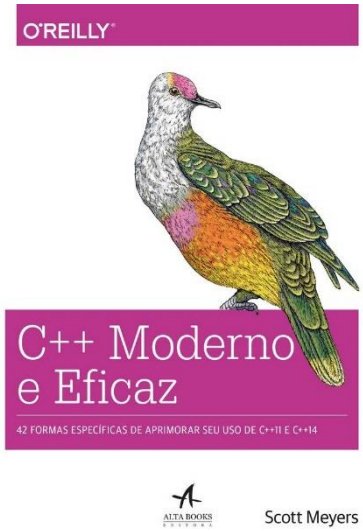
(2013)

Origem da linguagem C++ e aplicações

Diversos autores criaram seus livros de C++...







Origem da linguagem C++ e aplicações

- Em 1998, o comitê consultivo e de padrões de C++ publicou o primeiro padrão internacional para C++ ISO/IEC 14882:1998
- Conhecido informalmente como **C++98**
- Esse primeiro padrão acrescentava muitas coisas além do pensado e documentado por Bjarne em 1985
- Em 2003, o comitê publicou o segundo padrão de C++, denominado C++03.

Origem da linguagem C++ e aplicações

- O próximo padrão foi criado em 2011, denominado **C++11**

- Esse novo padrão trouxe significativas mudanças na linguagem, sendo este e seus sucessores denominados na comunidade de **C++ moderno**

- Outros padrões criados posteriormente:
  - C++14
  - C++17
  - C++20

INSTITUTO FEDERAL
Santa Catarina

DAELN

Introdução à linguagem C++

Forma geral de um programa em C++

Forma geral de um programa em C++

- Um **programa executável** é um arquivo com diversas instruções de máquina do **processador alvo**.

- Para criar um programa executável, usando o C++ como linguagem de desenvolvimento, precisamos pelo menos de:
  - ***toolchain*** (vou falar logo logo...);
  - Um arquivo no formato .cpp onde iremos colocar o código em formato C++

Forma geral de um programa em C++

- Será usada a versão C++11, pois é a que está disponível no ambiente de desenvolvimento que vamos usar

- Nessa versão, o programa em C++ tem como **ponto de entrada**, uma função denominada `main`, que tem os seguintes protótipos:

```
int main(void)
```

```
int main(int argc, char *argv[])
```

Forma geral de um programa em C++

ESQUELETO DE UM PROGRAMA EM C++

```
int main(void)
{
   // Os três pontos aqui é uma representação
   // Quer dizer que tem mais código aqui embaixo
   // Aliás, essas três linhas são comentários
   ...

   // No final do main, deve ser retornado um valor
   // para o SO (quem chamou o programa)
   // Se zero é OK! Se diferente de zero, então o SO
   // sabe que um problema deu no programa e pode
   // informar ao usuário
   return 0;
}
```

Forma geral de um programa em C++

REALIZANDO SOMA E ATRIBUIÇÃO

```
int main(void)
{
   // Abaixo, a declaração de uma variável
   int x;
   // Essa variável é do tipo int, ou seja, armazena
   // um número inteiro. Abaixo estamos colocando o
   // valor 6 dentro da variável.
   x = 6;
   // Abaixo estamos realizando a operação de soma
   // entre o valor guardado em x com o número 3, e
   // guardando esse resultado novamente em x
   x = x + 3;
   return 0;
}
```

Forma geral de um programa em C++

SAÍDA PADRÃO

```
// Devemos incluir a biblioteca onde está definida a
// operação std::cout
#include <iostream>

int main(void)
{
   int x;
   x = 6;
   x = x + 3;
   // Abaixo estamos pegando o valor guardado em x e
   // enviando à saída padrão.
   std::cout << x;
   return 0;
}
```

Forma geral de um programa em C++

SAÍDA PADRÃO

```
#include <iostream>
// Podemos omitir o std:: de std::cout se usarmos
// o comando abaixo
using namespace std;

int main(void)
{
   int x;
   x = 6;
   x = x + 3;
   // Abaixo estamos pegando o valor guardado em x e
   // enviando à saída padrão.
   cout << x;
   return 0;
}
```

Forma geral de um programa em C++

ENTRADA PADRÃO

```
// Devemos incluir a biblioteca onde está definida a
// operação std::cin
#include <iostream>
using namespace std;

int main(void)
{
   int x;
   // Abaixo estamos pegando o valor da entrada
   // padrão e guardado em x.
   cin >> x;

   return 0;
}
```

Forma geral de um programa em C++

ENTRADA PADRÃO COM MENSAGEM

```
// Devemos incluir a biblioteca onde está definida a
// operação std::cin
#include <iostream>
using namespace std;

int main(void)
{
   int x;
   // Podemos enviar mensagem ao usuário.
   cout << "Digite um valor: ";
   cin >> x;

   return 0;
}
```

Forma geral de um programa em C++

PASSAGEM DE PARÂMETROS POR TERMINAL

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
   cout << "Você digitou o seguinte: \n";
   // Abaixo, se tem um laço de repetição.
   // argv é um vetor de caracteres.
   // argc é a quantidade de caracteres dentro de argv
   // O laço imprime cada caractere seguindo de nova
   // linha a cada iteração do laço.
   for ( int i = 0; i < argc; ++i )
        cout << argv[i] << "\n";

   return 0;
}
```

INSTITUTO FEDERAL
Santa Catarina

DAELN

Introdução à linguagem C++

Etapas de desenvolvimento de um programa

Etapas de desenvolvimento de um programa

Como é feito um programa?

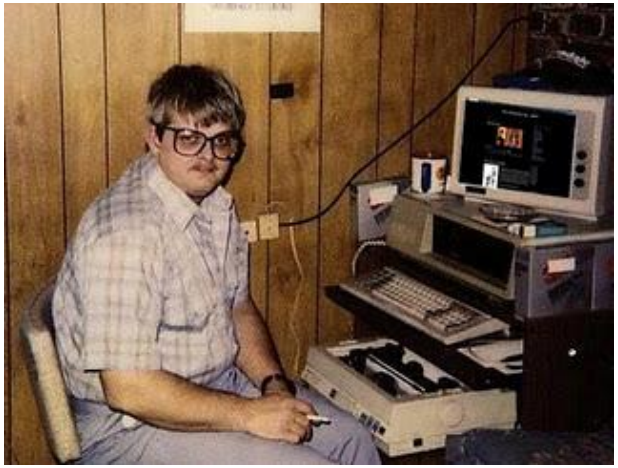
Memória de programa

00000000_H	11110000
00000001_H	01011110
00000002_H	11100010
00000003_H	00000110
00000004_H	11000111
00000005_H	11101001
00000006_H	01000001
00000007_H	10101000
...	...
$00003FFF_H$	10011011

Endereços

Programa descrito pelo programador

Programador

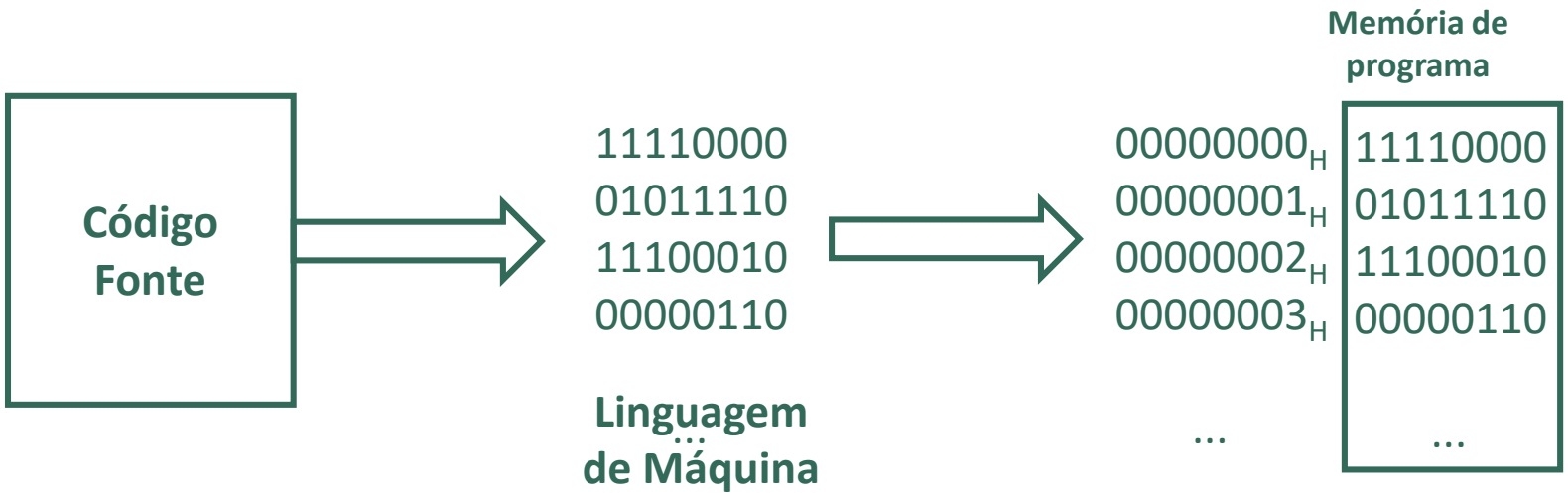


Como colocar o programa feito pelo programador dentro da memória de programa?

Como é descrito o programa pelo programador?

Etapas de desenvolvimento de um programa

- O programador descreve o programa através de uma **linguagem de programação**
- O programa descrito é denominado **código fonte** (arquivo de texto)
- O C++ é uma **linguagem compilada**, ou seja, o código fonte é traduzido na **linguagem de máquina** do processador



Etapas de desenvolvimento de um programa

- A linguagem de programação pode ser:
  - **Baixo nível**: denominada **linguagem assembly**. Linguagem própria do processador.
  - **Alto nível**: linguagem mais abstrata. Mais fácil entendimento para o ser humano. Existem diversas (ex.: Java, C, C++, Python, etc.)

```
;R0 parameter
factorial
PUSH {R1, R2, LR}
CMP R0, #1
BEQ ret
MOV R1, R0
SUB R0, R0, #1
BL factorial
MUL R2, R1, R0 ; int result = fact(n-1)*n
MOV R0, R2 ; R0 = result
POP {R1, R2, LR} ; return
ret
MOV R0, #1 ;return 1
POP {R1, R2, LR} ; return
```

Código fonte no assembly do processador ARM

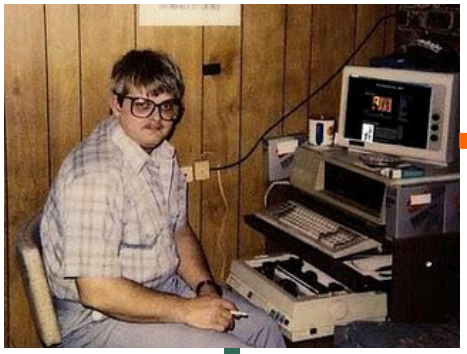
```
void  factorial(int n)
{
    int c, fact = 1;
    for (c = 1; c <= n; c++)
     fact = fact * c;

    return fact;
}
```

Código fonte equivalente em C++

Etapas de desenvolvimento de um programa

CONVERSÃO DE CÓDIGO EM LINGUAGEM DE ALTO PARA BAIXO NÍVEL



Programador

Programar direto em *assembly*

Programação em C++

```
void  factorial(int n)
{
   int c, fact = 1;
   for (c = 1; c <= n; c++)
    fact = fact * c;

   return fact;
}
```

Código fonte em C++

Compilador

```
;R0 parameter
factorial
PUSH {R1, R2, LR}
CMP R0, #1
BEQ ret
MOV R1, R0
SUB R0, R0, #1
BL factorial
MUL R2, R1, R0 ; int result = fact(n-1)*n
MOV R0, R2 ; R0 = result
POP {R1, R2, LR} ; return
ret
MOV R0, #1 ;return 1
POP {R1, R2, LR} ; return
```

Código fonte no assembly do processador ARM

Etapas de desenvolvimento de um programa

MONTAGEM

```
;R0 parameter
factorial
PUSH {R1, R2, LR}
CMP R0, #1
BEQ ret
MOV R1, R0
SUB R0, R0, #1
BL factorial
MUL R2, R1, R0 ; int result = fact(n-1)*n
MOV R0, R2 ; R0 = result
POP {R1, R2, LR} ; return
ret
MOV R0, #1 ;return 1
POP {R1, R2, LR} ; return
```

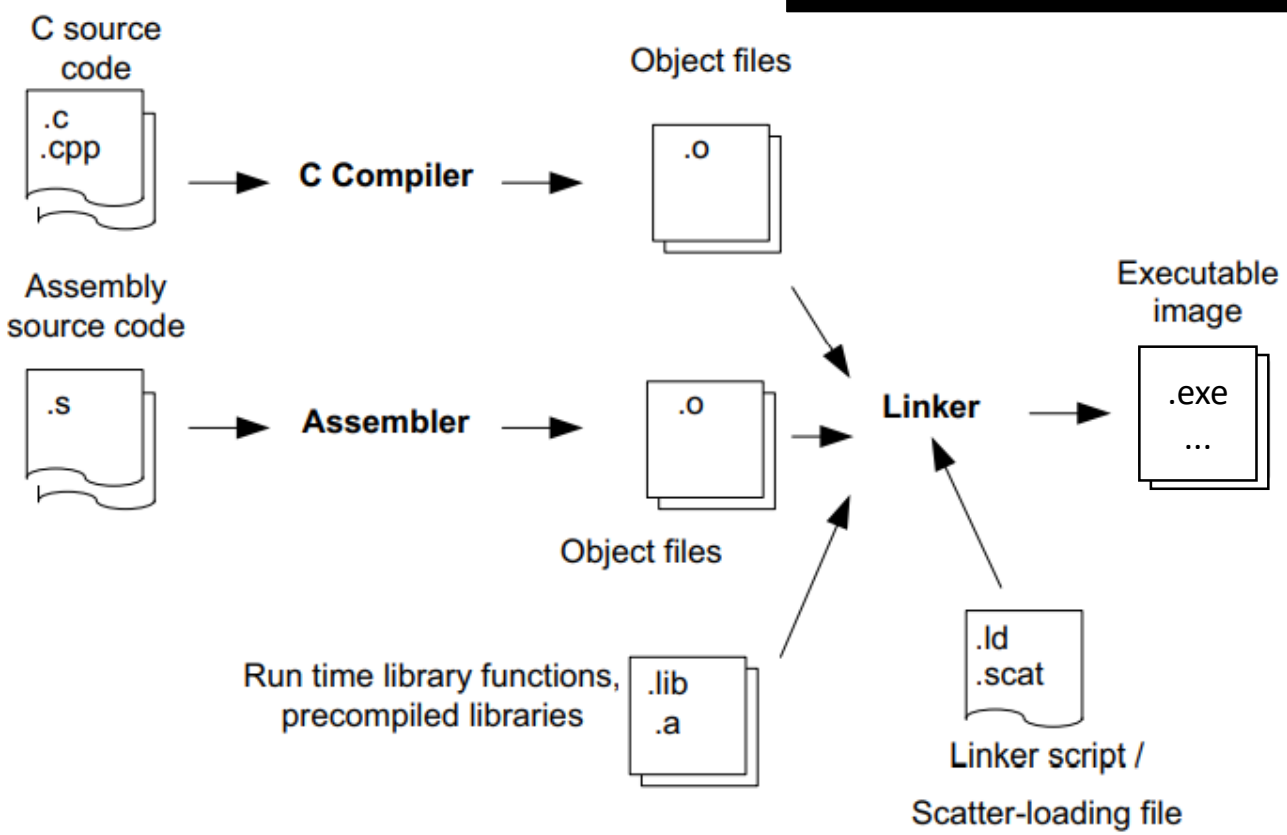
Montador (*Assembler*)

```
11110000
01011110
11100010
00000110
...
```

Código em linguagem de máquina + informações sobre o código (Código Objeto)

Etapas de desenvolvimento de um programa

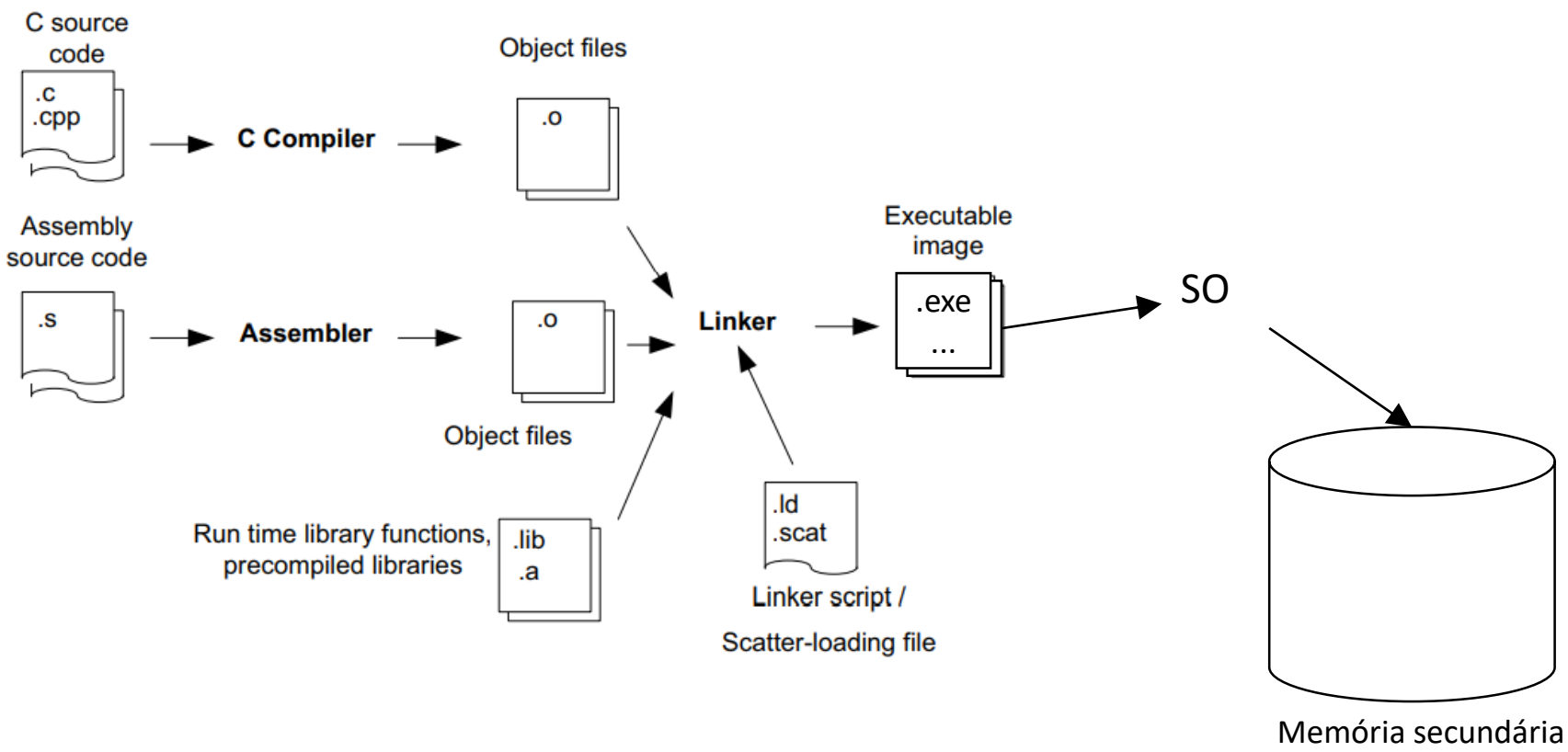
FLUXO DE GERAÇÃO DE EXECUTÁVEL



Modificado de YIU (2015)

Etapas de desenvolvimento de um programa

FLUXO DE GERAÇÃO ARMAZENAMENTO PELO SO

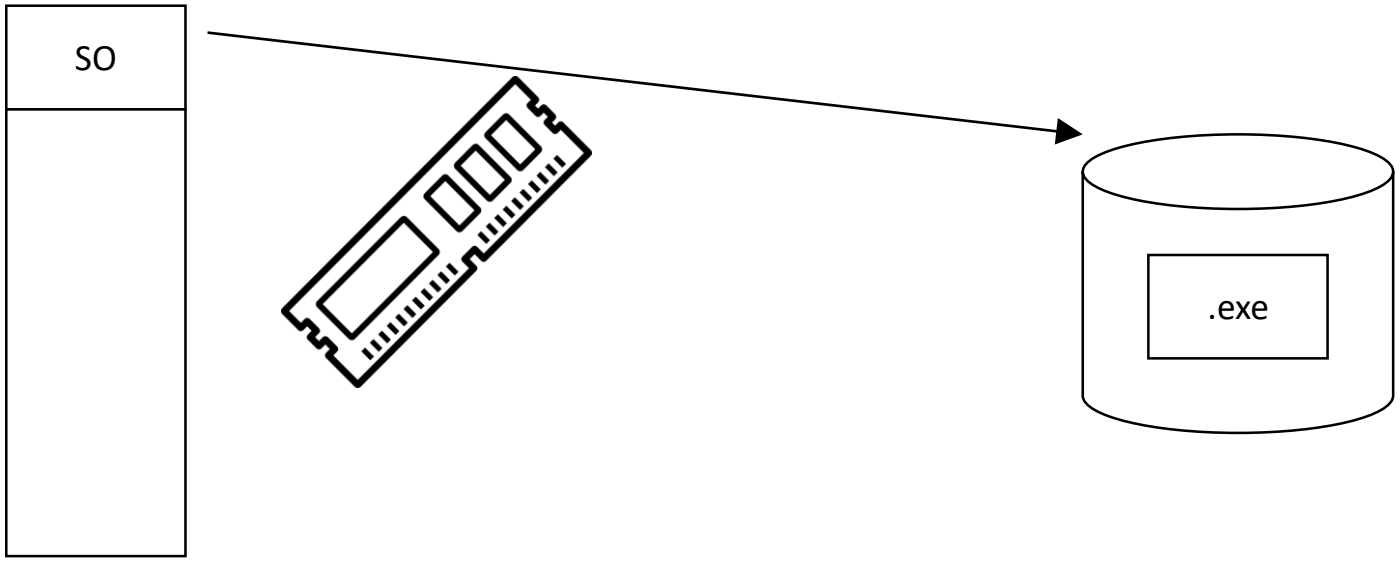


YIU (2015)

Etapas de desenvolvimento de um programa

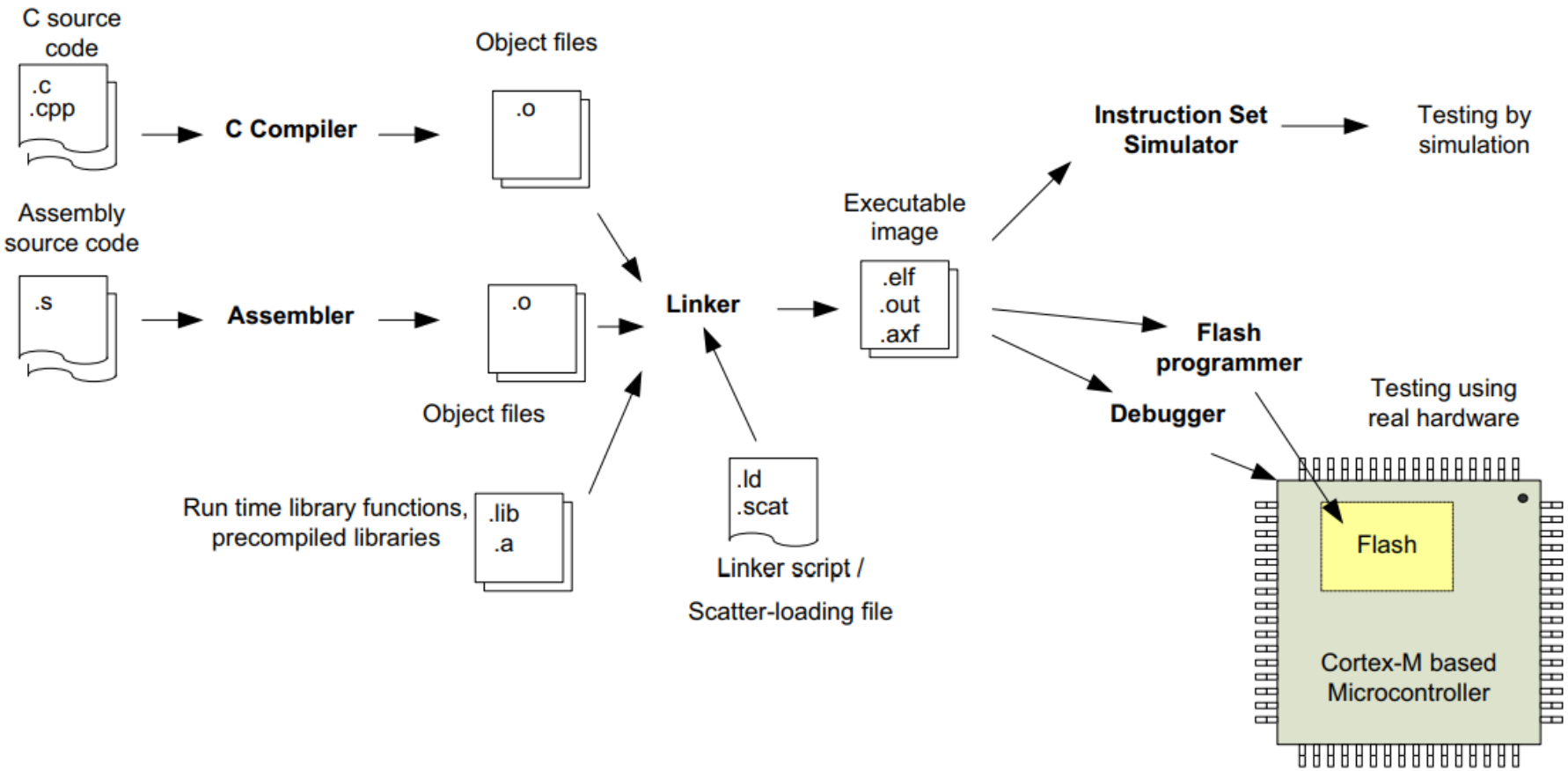
GARREGAMENTO DE EXECUTÁVEL PELO SO

Espaço da memória principal



Etapas de desenvolvimento de um programa

FLUXO DE GERAÇÃO DE FIRMWARE



YIU (2015)

Etapas de desenvolvimento de um programa

CONVERSÃO: LINGUAGEM DE ALTO PARA BAIXO NÍVEL

```
for(i=0; i<10;
++i){
    m=m+j+c;
    j=j-i;
}
```

Compilador C→MIPS

```
...
add $17, $18, $19
add $16, $16, $17
sub $20, $20, $21
...
```

Montador MIPS

```
...
00000100000001000000001
0000000100
00000101100100100000000
1000010100...
```

Processador MIPS R3000

Compilador C→SPARC

```
...
add %g4, %g1, %g2
add %g3, %g3, %g4
sub %g5, %g5, %g7
...
```

Montador SPARC

```
...
10000101100000001100000
1000010100
10000000001000000001001
0000000000...
```

Processador SPARC V8

Compilador C→IA32

```
...
add eax, ebx,
ecx
sub ax, bx, cx
...
```

Montador IA32

```
...
00000000010000000000000
0000000011
00000100100000000000001
1000010100...
```

Processador Pentium

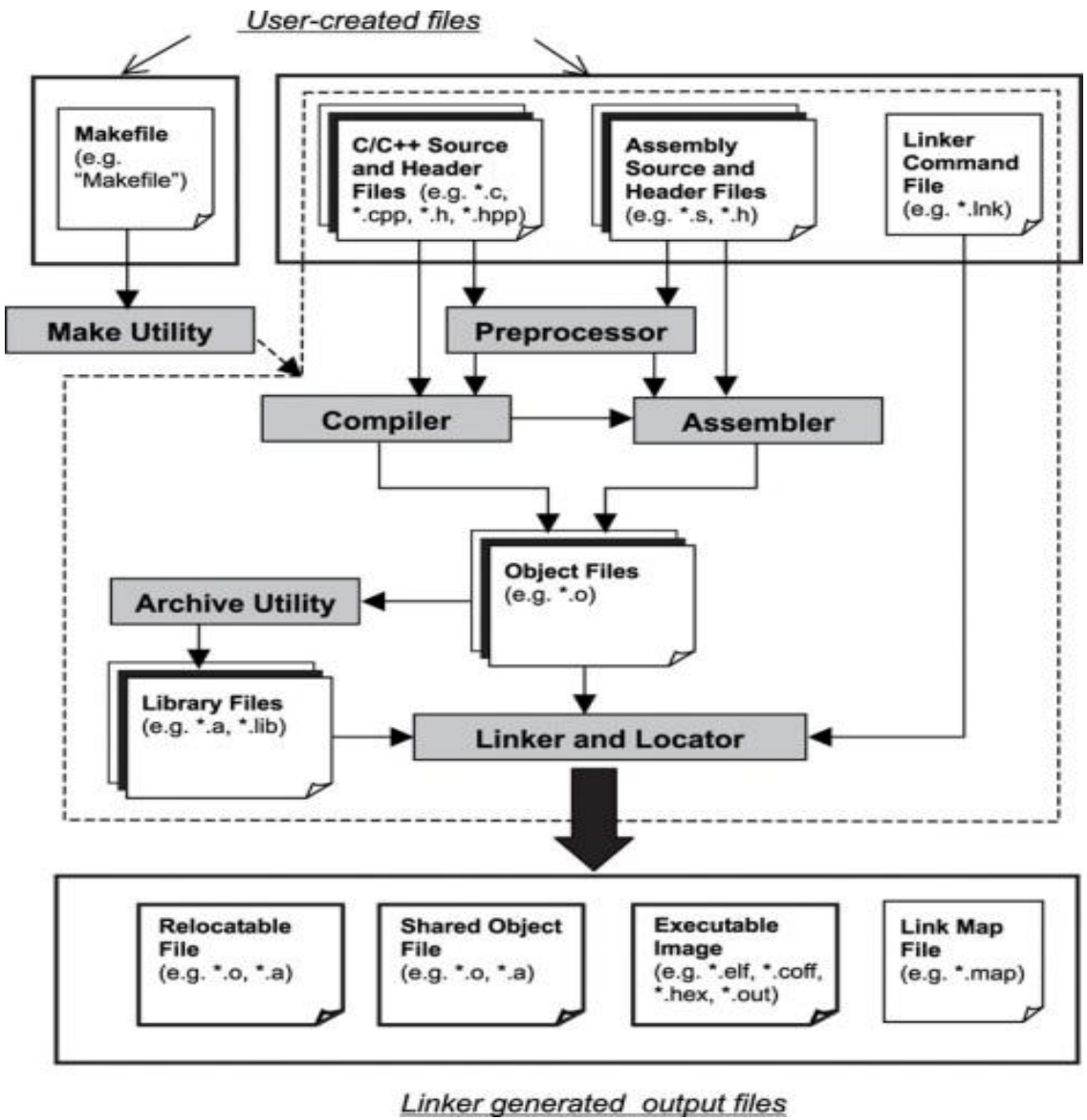
Etapas de desenvolvimento de um programa

- Uma *toolchain* bastante usada, seja por ser *open source*, ou por ser bastante consolidada no desenvolvimento de aplicações em C/C++ é o ***GNU toolchain***.

- Composta por:
  - GCC (GNU C/C++ Compiler)
  - GNU Binutils
  - Biblioteca C/C++
  - GDB (GNU Debugger)

Etapas de desenvolvimento de um programa

GNU ARM CROSS-TOOLCHAIN



Etapas de desenvolvimento de um programa

- Por onde começar a programar? Como achar o *toolchain* para o meu computador/linguagem?

- **Ambiente de Desenvolvimento Integrado**
- Do inglês: ***Integrated Development Environment*** (**IDE**)

INSTITUTO FEDERAL
Santa Catarina

DAELN

Introdução à linguagem C++

Conteúdo programado

Conteúdo programado

1 – INTRODUÇÃO

- Introdução a linguagem C++.
- Origem da linguagem e aplicações.
- Forma geral de um programa em C++.
- Etapas de desenvolvimento de um programa.
- Compilação, ligação e execução de programas.

Conteúdo programado

2 – EXPRESSÕES E VARIÁVEIS

- Tipos de dados básicos em C++
- Declaração e Inicialização de variáveis
- Constantes
- Expressões aritméticas
- Expressões relacionais
- Expressões lógicas
- Expressões compostas
- *Casting*

Conteúdo programado

3 – CONTROLE DE FLUXO DE EXECUÇÃO

- Estruturas de controle condicional
  - if-else
  - switch
- Estruturas de repetição
  - while
  - do-while
  - for

Conteúdo programado

4 – FUNÇÕES

- Conceitos básicos
- Declaração e definição de funções
- Escopo de variáveis
- Passagem de parâmetros por valor e referência
- Algumas funções da biblioteca padrão

Conteúdo programado

5 – INTRODUÇÃO A CLASSES E OBJETOS

- Classes e objetos
- Atributos e métodos
- Construtores e destrutores
- Encapsulamento

Conteúdo programado

6 – VETORES E MATRIZES

- Inicialização de vetores
- Indexação de vetores
- Matrizes

Conteúdo programado

7 – STRINGS

- Formato de uma string
- Inicialização de strings
- Biblioteca `cstring`
- Classe `string`

Conteúdo programado

8 – STRUCTS E UNIONS

- Structs
- Unions
- Bit fields

Conteúdo programado

9 – OPERAÇÕES BITWISE

- Máscara de bits
- Operação AND bit-a-bit
- Operação OR bit-a-bit
- Operação NOT bit-a-bit
- Deslocamento bit-a-bit
- Aplicação de mascaras

Conteúdo programado

10 – PONTEIROS

- Conceito de ponteiro.
- Operadores de endereço e diferenciação.
- Ponteiro para ponteiros.
- Passagem de parâmetros por referência.

Conteúdo programado

11 – ALOCAÇÃO DINÂMICA DE MEMÓRIA

- Alocação estática versus alocação dinâmica.
- Funções de alocação dinâmica de memória.

INSTITUTO FEDERAL
Santa Catarina

DAELN

Introdução à linguagem C++

Avaliação na disciplina

Avaliação na disciplina

- A nota final será dada conforme seguinte equação:

$$N = [(T1+T2)/2]*0,4 + [(P1+P2)/2]*0,6$$

- Onde:
  - T1 e P1 são as provas de cunho teórico e prático sobre a primeira parte do conteúdo;
  - T2 e P2 são as provas de cunho teórico e prático sobre a segunda parte do conteúdo.

Avaliação na disciplina

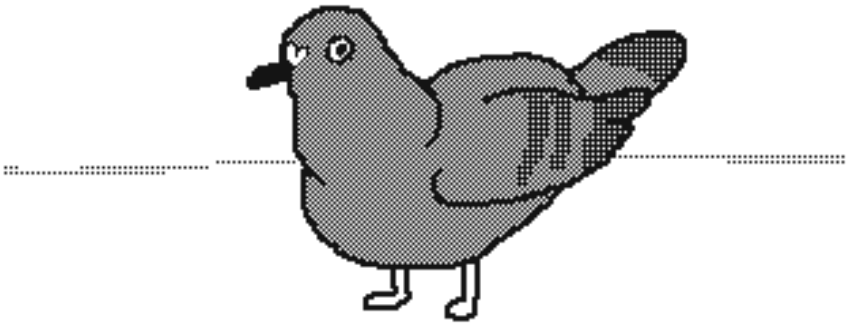
- Caso o aluno não obtenha a média para aprovação na disciplina, será oportunizado:
  - Uma prova de cunho teórico sobre todo o conteúdo da disciplina, que será substitutiva à média das provas teóricas, caso maior;
  - Uma prova de cunho prático sobre todo o conteúdo da disciplina, que será substitutiva à média das provas práticas, caso maior.

Avaliação na disciplina

- As suas atividades práticas de programação serão avaliadas conforme os seguintes critérios:
  - Qualidade
  - Indentação e
  - Documentação

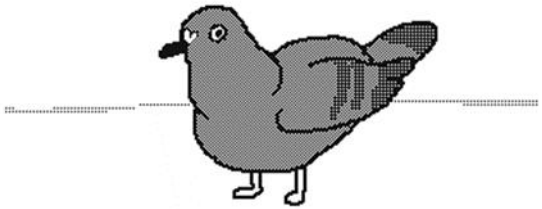
Avaliação na disciplina

- Quando meu código não tem qualidade?



Avaliação na disciplina

- O que deixa meu código sem qualidade?
  - **Código inatingível** - código nunca executado, desperdiçando memória e inserindo linhas de código que atrapalham a correção.(ex.: funções nunca chamadas, variáveis declaradas, mas não usadas)
  - **Código morto** - código que quando executado, não acrescenta mudança ao programa (ex.: if (true){...})).
  - Variáveis não inicializadas, mas seu valor deve ser usado logo na inicialização.
  - Acesso fora dos limites do *array*.
  - Etc.



Avaliação na disciplina

- Documentação

MAU DOCUMENTADO

```
int main(void)
{
   int x;
   x = 6;
   x = x + 3;
   return 0;
}
```

BEM DOCUMENTADO

```
int main(void)
{
   // O valor x é usado como exemplo para
   // verificação das operações de
   // atribuição e operação aritmética
   int x;
   x = 6;
   x = x + 3;
   return 0;
}
```

MAU DOCUMENTADO

```
// Chamada da função main
int main(void)
{
   // Abaixo, a declaração de uma variável
   int x;
   x = 6; // Valor x atribuído ao 6
   x = x + 3; // Realiza a operação x = x + 3
   return 0; // Retorna o valor 0 para o sistema
}
```

Avaliação na disciplina

- Indentação

BEM INDENTADO

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
   for ( int i = 0; i < argc; ++i )
        cout << argv[i] << "\n";

   return 0;
}
```

MAU INDENTADO

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
for ( int i = 0; i < argc; ++i )
cout << argv[i] << "\n";

return 0;
}
```